

GAAuth P*P Architecture User Guide

Contents

1	GAAuth P*P Architecture User Guide	2
1.1	Table of Contents	2
1.2	Introduction	2
1.3	PAP - Policy Administration Point	3
1.4	PDP - Policy Decision Point	8
1.5	PIP - Policy Information Point	12
1.6	PVP - Policy Verification Point	15
1.7	PEP - Policy Enforcement Point	19
1.8	Integration Examples	23
1.9	Best Practices	25
1.10	Quick Reference	26
1.11	Additional Resources	27

Chapter 1

GAuth P*P Architecture User Guide

Complete Guide to Policy Administration, Decision, Information, Verification, and Enforcement Points

1.1 Table of Contents

1. [Introduction](#)
 2. [PAP - Policy Administration Point](#)
 3. [PDP - Policy Decision Point](#)
 4. [PIP - Policy Information Point](#)
 5. [PVP - Policy Verification Point](#)
 6. [PEP - Policy Enforcement Point](#)
 7. [Integration Examples](#)
 8. [Best Practices](#)
-

1.2 Introduction

The ****P*P Architecture**** (pronounced “P-Star-P”) is the core authorization framework of GAuth 1.0, implementing RFC-0111 compliant policy-based access control. It consists of five interoperating components:

- **PAP** (Policy Administration Point) - Creates and manages authorization policies
- **PDP** (Policy Decision Point) - Makes authorization decisions based on policies
- **PIP** (Policy Information Point) - Retrieves contextual attributes for decisions
- **PVP** (Policy Verification Point) - Verifies identity and authorization chains
- **PEP** (Policy Enforcement Point) - Enforces authorization decisions

Authorization Flow

PEP	Intercepts Request
Enforcement	

PVP	PIP	PDP
Verify	Retrieve	Decision
Identity	Attributes	Making

PAP	Policy Storage
Admin Point	

1.3 PAP - Policy Administration Point

1.3.1 What is PAP?

The **Policy Administration Point** is responsible for creating, managing, and maintaining authorization policies. It provides a complete policy lifecycle management system with versioning, activation, suspension, and revocation capabilities.

1.3.2 Key Features

- **Policy CRUD Operations** - Create, Read, Update, Delete policies
- **Lifecycle Management** - Draft → Active → Suspended → Revoked
- **Policy Types** - PoA, Authorization Chain, Scope, Restriction, Compliance
- **Search & Filtering** - Find policies by type, status, owner, tags
- **Validation** - Comprehensive policy validation before activation
- **Statistics** - Aggregate metrics across all policies
- **Thread-Safe** - Concurrent policy access with mutex protection

1.3.3 Policy Lifecycle States

DRAFT Created, can be edited

Activate (with validation)

ACTIVE Enforced, cannot be edited

Suspend (temporary) or Revoke (permanent)

SUSPENDED REVOKED

Reactivate Delete

ACTIVE DELETED

1.3.4 Usage Examples

1.3.4.1 1. Create a Policy

```
package main

import (
    "context"
    "fmt"
    "github.com/mauriciomferz/Gauth_go/pkg/gauth"
)

func main() {
    // Initialize PAP
    pap := gauth.NewPowerAdministrationPoint(
        "pap-001",
        "Production PAP",
        "Main policy administration point for GAuth",
    )

    ctx := context.Background()

    // Create a Power of Attorney policy
    createRequest := &gauth.PolicyCreateRequest{
        PolicyType:      gauth.PolicyTypePoA,
        PolicyName:      "Healthcare Data Access Policy",
        Description:     "Allows healthcare providers to access patient records",
        ClientOwner:     "hospital-client-001",
        OwnersAuthorizer: "ceo-authorizer-001",

        PolicyRules: gauth.PolicyRules{
            AllowedActions: []string{"read", "write", "update"},
            ResourcePatterns: []string{"/api/patients/*", "/api/records/*"},
            DeniedActions: []string{"delete"},
        }
    }
```

```

    },

    Scope: &gauth.PolicyScope{
        Countries: []string{"US", "CA"},
        Sectors:    []string{"healthcare"},
        Regions:    []string{"US-CA", "US-NY"},
    },

    Tags: []string{"healthcare", "production", "gdpr-compliant"},

    Metadata: map[string]interface{}{
        "department": "Healthcare IT",
        "cost_center": "HC-001",
        "compliance_level": "high",
    },
}

policy, err := pap.CreatePolicy(ctx, createRequest)
if err != nil {
    fmt.Printf("Failed to create policy: %v\n", err)
    return
}

fmt.Printf(" Policy Created: %s\n", policy.PolicyID)
fmt.Printf("   Name: %s\n", policy.PolicyName)
fmt.Printf("   Status: %s\n", policy.Status) // "draft"
fmt.Printf("   Version: %d\n", policy.PolicyVersion)
}

```

1.3.4.2 2. Activate a Policy

```

// Validate and activate the policy
err := pap.ActivatePolicy(ctx, policy.PolicyID, "approver-cto-001")
if err != nil {
    fmt.Printf("Activation failed: %v\n", err)
    return
}

fmt.Println(" Policy activated and enforced")

```

1.3.4.3 3. Update a Policy (Draft Only)

```

updateRequest := &gauth.PolicyUpdateRequest{
    PolicyName: ptr("Updated Healthcare Policy Name"),
    Description: ptr("Updated description"),
    PolicyRules: &gauth.PolicyRules{
        AllowedActions: []string{"read", "write", "update", "export"},
    },
}

```

```

    Tags: []string{"healthcare", "production", "updated"},
}

updatedPolicy, err := pap.UpdatePolicy(ctx, policy.PolicyID, updateRequest)
if err != nil {
    fmt.Printf("Update failed: %v\n", err)
    return
}

fmt.Printf(" Policy updated to version %d\n", updatedPolicy.PolicyVersion)

```

1.3.4.4 4. Search Policies

```

// Search for healthcare policies
criteria := &gauth.PolicySearchCriteria{
    PolicyType: ptr(gauth.PolicyTypePoA),
    Status:     ptr(gauth.PolicyStatusActive),
    Tags:       []string{"healthcare"},
    Countries:  []string{"US"},
}

policies, err := pap.SearchPolicies(ctx, criteria)
if err != nil {
    fmt.Printf("Search failed: %v\n", err)
    return
}

fmt.Printf("Found %d matching policies:\n", len(policies))
for _, p := range policies {
    fmt.Printf(" - %s (%s)\n", p.PolicyName, p.PolicyID)
}

```

1.3.4.5 5. Get Policy Statistics

```

stats, err := pap.GetPolicyStatistics(ctx)
if err != nil {
    fmt.Printf("Failed to get statistics: %v\n", err)
    return
}

fmt.Printf(" Policy Statistics:\n")
fmt.Printf("   Total: %d\n", stats.TotalPolicies)
fmt.Printf("   Active: %d\n", stats.ActivePolicies)
fmt.Printf("   Draft: %d\n", stats.DraftPolicies)
fmt.Printf("   Suspended: %d\n", stats.SuspendedPolicies)
fmt.Printf("   Revoked: %d\n", stats.RevokedPolicies)

for policyType, count := range stats.PoliciesByType {

```

```
    fmt.Printf("    %s: %d\n", policyType, count)
}
```

1.3.4.6 6. Policy Lifecycle Management

```
// Suspend a policy temporarily
err = pap.SuspendPolicy(ctx, policy.PolicyID, "Maintenance window")
// Policy status → SUSPENDED

// Reactivate suspended policy
err = pap.ActivatePolicy(ctx, policy.PolicyID, "approver-001")
// Policy status → ACTIVE

// Revoke policy permanently
err = pap.RevokePolicy(ctx, policy.PolicyID, "Policy no longer needed")
// Policy status → REVOKED

// Delete revoked policy
err = pap.DeletePolicy(ctx, policy.PolicyID)
// Policy removed from storage
```

1.3.5 REST API Endpoints

PAP can also be accessed via REST API:

```
# Create Policy
POST /api/v1/pap/policies
Content-Type: application/json

{
  "policy_type": "poa",
  "policy_name": "New Policy",
  "client_owner": "client-001",
  "owners_authorizer": "auth-001",
  "policy_rules": {
    "allowed_actions": ["read", "write"]
  }
}

# Get Policy
GET /api/v1/pap/policies/:policy_id

# Update Policy
PUT /api/v1/pap/policies/:policy_id

# Activate Policy
POST /api/v1/pap/policies/:policy_id/activate

# Search Policies
```



```
GET /api/v1/pap/policies/search?type=poa&status=active
```

```
# Get Statistics
```

```
GET /api/v1/pap/statistics
```

1.3.6 Implementation Details

- **Location:** [pkg/gauth/gauth.go](#) (1,279+ lines)
 - **Tests:** [pkg/gauth/pap_test.go](#) (13 test suites, 97.8% coverage)
 - **Types:** [pkg/gauth/pap_types.go](#)
 - **API Routes:** [web/server_clean.go](#) (12 endpoints)
-

1.4 PDP - Policy Decision Point

1.4.1 What is PDP?

The **Policy Decision Point** evaluates authorization requests against active policies and makes access control decisions. It supports RBAC (Role-Based Access Control), ABAC (Attribute-Based Access Control), and hybrid authorization models.

1.4.2 Key Features

- **Multi-Engine Support** - RBAC, ABAC, hybrid evaluation
- **Policy Combination** - Deny-overrides, permit-overrides, first-applicable
- **Caching** - Distributed decision cache for performance
- **PAP Integration** - Direct integration with PAP for policy retrieval
- **Obligations** - Post-decision requirements (logging, notifications)
- **Context-Aware** - Evaluates environmental attributes (time, location, IP)

1.4.3 Decision Flow

```
Authorization Request  
{subject, resource, action, context}
```

```
1. Load Applicable Policies from PAP
```

```
2. Evaluate Policies (RBAC/ABAC)
```

- Match targets
- Evaluate conditions
- Apply combining algorithms

3. Make Decision
→ PERMIT / DENY / NOT_APPLICABLE

4. Process Obligations
- Logging, notifications, etc.

Decision
Response

1.4.4 Usage Examples

1.4.4.1 1. Create PDP with PAP Integration

```
package main

import (
    "context"
    "fmt"
    "github.com/mauriciomferz/Gauth_go/pkg/gauth"
)

func main() {
    // Create PAP
    pap := gauth.NewPowerAdministrationPoint("pap-001", "Production PAP", "")

    // Create PDP with PAP integration
    pdp := gauth.NewSimplePDPWithPAP(pap)

    fmt.Println(" PDP initialized with PAP integration")
}
```

1.4.4.2 2. Make Authorization Decision

```
// Create authorization request
request := &gauth.AuthorizationDecisionRequest{
    Subject: "user:alice@example.com",
    Resource: "/api/patients/12345",
    Action: "read",
}
```

```

Context: map[string]interface{}{
    "ip_address": "192.168.1.100",
    "time":      time.Now(),
    "location":  "US-CA",
    "role":      "doctor",
    "department": "cardiology",
},
}

// Evaluate authorization
decision, err := pdp.MakeDecision(context.Background(), request)
if err != nil {
    fmt.Printf("Decision error: %v\n", err)
    return
}

fmt.Printf("Decision: %s\n", decision.Decision) // "allow" or "deny"
fmt.Printf("Reason: %s\n", decision.Reason)

if decision.Decision == "allow" {
    fmt.Println(" Access GRANTED")
    // Proceed with the request
} else {
    fmt.Println(" Access DENIED")
    // Reject the request
}

```

1.4.4.3 3. Distributed PDP with Caching

```

import "github.com/mauriciomferz/Gauth_go/internal/pdp"

// Create distributed PDP with caching
config := &pdp.PDPConfig{
    NodeID:      "pdp-node-001",
    Address:     "192.168.1.10:8080",
    CacheTTL:    5 * time.Minute,
    CacheMaxSize: 10000,
    HealthCheckInterval: 30 * time.Second,
    ClusterSyncInterval: 1 * time.Minute,
}

distributedPDP, err := pdp.NewDistributedPDP(config)
if err != nil {
    log.Fatal(err)
}

// Start PDP

```

```

err = distributedPDP.Start(context.Background())
if err != nil {
    log.Fatal(err)
}
defer distributedPDP.Stop()

// Make decision (uses cache when available)
decisionReq := &pdp.DecisionRequest{
    RequestID: "req-12345",
    Subject:   "user:alice",
    Action:    "read",
    Resource:   "document:secret",
    Context:   map[string]interface{}{
        "ip": "10.0.0.1",
    },
    CacheTTL: 5 * time.Minute,
}

response, err := distributedPDP.MakeDecision(ctx, decisionReq)
fmt.Printf("Decision: %s (from node: %s)\n", response.Decision, response.NodeID)

```

1.4.4.4 4. Add/Remove Policies via PDP

```

// Add policy through PDP (uses PAP internally)
policy := &gauth.AuthorizationPolicy{
    PolicyName:      "API Access Policy",
    PolicyType:      gauth.PolicyTypePoA,
    ClientOwner:     "api-team",
    OwnersAuthorizer: "tech-lead",
    PolicyRules: gauth.PolicyRules{
        AllowedActions: []string{"read", "write"},
    },
    Scope: &gauth.PolicyScope{
        Countries: []string{"US"},
    },
}

err := pdp.AddPolicy("", policy)
if err != nil {
    fmt.Printf("Failed to add policy: %v\n", err)
}

// Activate policy via PAP
err = pap.ActivatePolicy(ctx, policy.PolicyID, "approver-001")

// List active policies
activePolicies, err := pdp.ListActivePolicies()
fmt.Printf("Active policies: %d\n", len(activePolicies))

```

1.4.5 REST API Endpoints

Make authorization decision

POST /api/v1/pdp/authorize

Content-Type: application/json

```
{
  "subject": "user:alice",
  "resource": "/api/data",
  "action": "read",
  "context": {
    "ip_address": "192.168.1.1",
    "time": "2025-11-17T10:00:00Z"
  }
}
```

Response

```
{
  "decision": "allow",
  "reason": "Policy PoA-001 matched",
  "obligations": ["log_access"],
  "matched_policies": ["policy-xyz-123"]
}
```

1.4.6 Implementation Details

- **Location:** `pkg/gauth/pdp_adapter.go` (SimplePDP)
 - **Distributed:** `internal/pdp/distributed_pdp.go`
 - **Decision Types:** DecisionPermit, DecisionDeny, DecisionNotApplicable, DecisionIndeterminate
-

1.5 PIP - Policy Information Point

1.5.1 What is PIP?

The **Policy Information Point** retrieves contextual attributes and information needed for policy evaluation. It integrates with external data sources like identity providers, registries, and databases.

1.5.2 Key Features

- **Multi-Source Integration** - PVP (identity), Registry (entities), PoA (delegations)
- **Attribute Caching** - Reduces latency with configurable TTL
- **Real-Time Retrieval** - Fresh data for each authorization decision
- **Geographic Validation** - ISO 3166-1/3166-2 compliant location checks
- **18-Country Support** - Identity connectors for global coverage

1.5.3 Data Sources

Policy Information Point

Registry (Entities)	PVP (Identity)	PoA (Delegations)
------------------------	-------------------	----------------------

Company Info	Identity Proof	Active Powers
Directors	Verification	Action Limits
Signatories	Entity Type	Validity

1.5.4 Usage Examples

1.5.4.1 1. Initialize PIP Client

```
package main

import (
    "github.com/mauriciomferz/Gauth_go/pkg/pip"
)

func main() {
    // Create PIP client
    pipClient := pip.NewClient(pip.Config{
        RegistryURL:    "http://localhost:8080",
        IdentityURL:    "http://localhost:8080",
        PolicyStoreURL: "http://localhost:8080",
        CacheTTL:      5 * time.Minute,
    })

    fmt.Println(" PIP client initialized")
}
```

1.5.4.2 2. Retrieve Subject Attributes

```
// Get user/subject attributes
subjectAttrs, err := pipClient.GetSubjectAttributes(ctx, "user:alice@example.com")
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Subject Attributes:\n")
fmt.Printf("  User ID: %v\n", subjectAttrs["user_id"])
```

```

fmt.Printf("  Roles: %v\n", subjectAttrs["roles"])
fmt.Printf("  Department: %v\n", subjectAttrs["department"])
fmt.Printf("  Clearance Level: %v\n", subjectAttrs["clearance_level"])

```

1.5.4.3 3. Retrieve Resource Attributes

```

// Get resource attributes
resourceAttrs, err := pipClient.GetResourceAttributes(ctx, "document:secret-123")
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Resource Attributes:\n")
fmt.Printf("  Classification: %v\n", resourceAttrs["classification"])
fmt.Printf("  Owner: %v\n", resourceAttrs["owner"])
fmt.Printf("  Created: %v\n", resourceAttrs["created"])

```

1.5.4.4 4. Verify Entity via Registry

```

// Verify company in commercial register
companyInfo, err := pipClient.VerifyEntity(ctx, "HRB-12345-DE", "DE")
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Company: %s\n", companyInfo.LegalName)
fmt.Printf("Status: %s\n", companyInfo.Status)
fmt.Printf("Directors: %d\n", len(companyInfo.ManagingDirectors))

```

1.5.4.5 5. Verify Identity via PVP

```

// Verify person identity
identityProof, err := pipClient.VerifyIdentity(ctx, "person-123", "eIDAS")
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Identity Verified: %v\n", identityProof.Verified)
fmt.Printf("Verification Level: %s\n", identityProof.Level)
fmt.Printf("Entity Type: %s\n", identityProof.EntityType)

```

1.5.5 Integration with PDP

```

// PDP automatically uses PIP for attribute enrichment
pdp := pdp.NewEngine(policyStore, keyProvider, pdp.WithPIP(pipClient))

// Authorization request - PIP fetches attributes automatically
request := &pdp.Request{
    Subject: "user:alice",

```

```

Resource: "document:123",
Action:   "read",
Context:  map[string]interface{}{
    "ip_address": "192.168.1.100",
    "time":       time.Now(),
},
}

// PIP enriches request with:
// - Subject attributes (roles, department, etc.)
// - Resource attributes (owner, classification, etc.)
// - Environmental attributes (location, time zone, etc.)
result := pdp.Evaluate(request)

```

1.5.6 REST API Endpoints

```

# Get subject attributes
GET /api/v1/pip/subjects/:subject_id/attributes

# Get resource attributes
GET /api/v1/pip/resources/:resource_id/attributes

# Verify entity
POST /api/v1/beta/registry/verify-entity

# Verify identity
POST /api/v1/beta/pvp/verify

```

1.6 PVP - Policy Verification Point

1.6.1 What is PVP?

The **Policy Verification Point** verifies identities, authorization chains, and trust relationships. It integrates with identity providers and trust services to validate claims.

1.6.2 Key Features

- **Identity Verification** - eIDAS, national ID systems (18 countries)
- **Trust Chain Validation** - Verifies authorization delegation chains
- **Multi-Method Support** - eIDAS, government ID, biometric, document
- **Geographic Coverage** - US, DE, UK, FR, IT, ES, SE, NL, AE, SA, JP, AU, SG, KR, IN, NZ, BR, CA, MX, ZA, NG, KE
- **Proof Storage** - Maintains verification evidence

1.6.3 Verification Methods

Method	Description	Countries
eIDAS	EU Digital Identity Framework	EU (27 countries)
Government ID	National ID cards	All 18 supported
Passport	International travel documents	Global
Biometric	Fingerprint, facial recognition	Select countries
Document	Driver's license, utility bills	All

1.6.4 Usage Examples

1.6.4.1 1. Verify Natural Person

```
package main

import (
    "context"
    "github.com/mauriciomferz/Gauth_go/pkg/gauth"
)

func main() {
    // Create PVP client (mock for development)
    pvpClient := gauth.NewMockPVPCClient(false)

    ctx := context.Background()

    // Verify natural person identity
    identityProof, err := pvpClient.VerifyIdentity(ctx, gauth.IdentityVerificationRequest{
        SubjectID:      "person-alice-001",
        IdentityType:    gauth.IdentityTypeNaturalPerson,
        ProofMethod:     gauth.ProofMethodEIDAS,
        Jurisdiction:    "DE",
        ProofData: map[string]interface{}{
            "verified":      true,
            "eidas_level":   "high",
            "document_id":   "DE-ID-123456",
            "issued_by":     "German Federal Office",
        },
        RequiredLevel:    "high",
    })

    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf(" Identity Verified\n")
    fmt.Printf("   Subject: %s\n", identityProof.SubjectID)
    fmt.Printf("   Type: %s\n", identityProof.EntityType)
    fmt.Printf("   Verified: %v\n", identityProof.Verified)
}
```

```

    fmt.Printf("    Level: %s\n", identityProof.Level)
    fmt.Printf("    Method: %s\n", identityProof.Method)
}

```

1.6.4.2 2. Verify Legal Entity

```

// Verify legal entity (company)
identityProof, err := pvpClient.VerifyIdentity(ctx, gauth.IdentityVerificationRequest{
    SubjectID:    "company-acme-001",
    IdentityType: gauth.IdentityTypeLegalEntity,
    ProofMethod:  gauth.ProofMethodCommercialRegister,
    Jurisdiction: "DE",
    ProofData: map[string]interface{}{
        "registration_number": "HRB-12345-DE",
        "legal_name":          "Acme Corporation GmbH",
        "verified":            true,
    },
    RequiredLevel: "high",
})

fmt.Printf("Company Verified: %v\n", identityProof.Verified)

```

1.6.4.3 3. Verify Authorization Chain

```

// Verify complete authorization chain
chainValid, err := pvpClient.VerifyAuthorizationChain(ctx, gauth.AuthorizationChainRequest{
    ChainID: "chain-001",
    Steps: []gauth.ChainStep{
        {
            Subject:    "company-acme",
            Authorizer: "director-001",
            Level:     "managing_director",
        },
        {
            Subject:    "director-001",
            Authorizer: "ceo-001",
            Level:     "delegation",
        },
    },
})

fmt.Printf("Chain Valid: %v\n", chainValid)

```

1.6.4.4 4. Verify with Multiple Methods

```

// Try multiple verification methods
verificationResult, err := pvpClient.VerifyWithFallback(ctx, "user-bob", []string{
    "eidas",
    "government_id",
})

```

```

        "passport",
        "document",
    })

    fmt.Printf("Verified with method: %s\n", verificationResult.Method)
    fmt.Printf("Confidence: %.2f\n", verificationResult.Confidence)

```

1.6.5 REST API Endpoints

Verify identity

POST /api/v1/beta/pvp/verify

Content-Type: application/json

```

{
  "subject_id": "person-alice",
  "identity_type": "natural_person",
  "proof_method": "eIDAS",
  "jurisdiction": "DE",
  "proof_data": {
    "verified": true,
    "eidas_level": "high"
  },
  "required_level": "high"
}

# Response
{
  "success": true,
  "verified": true,
  "identity_proof": {
    "subject_id": "person-alice",
    "entity_type": "natural_person",
    "verified": true,
    "level": "high",
    "method": "eIDAS",
    "verification_time": "2025-11-17T10:00:00Z"
  }
}

```

1.6.6 Implementation Details

- **Location:** Referenced in [pkg/rfc/combined_config.go](#)
- **Integration:** 18-country identity connectors in [docs/EXTERNAL_CONNECTORS_INTEGRATION_GUIDE.md](#)

1.7 PEP - Policy Enforcement Point

1.7.1 What is PEP?

The **Policy Enforcement Point** intercepts requests, validates tokens, consults PDP for authorization decisions, and enforces access control. It's the front-line gatekeeper for all authorization operations.

1.7.2 Key Features

- **Request Interception** - HTTP middleware, API gateway integration
- **Token Validation** - JWT/JWE extended token verification
- **PDP Consultation** - Real-time authorization decisions
- **Audit Logging** - Complete enforcement action tracking
- **Compliance Tracking** - Monitors authorization behavior
- **Enforcement Modes** - Strict (blocking) or Advisory (logging only)

1.7.3 Enforcement Flow

1. Request arrives with token
2. PEP intercepts request
3. Validate Token (JWT/JWE)
 - Extract claims
 - Verify signature
 - Check expiration
4. Consult PDP for Decision
 - Pass subject, resource, action
 - Include context attributes
5. Enforce Decision
 - ALLOW → Forward request
 - DENY → Return 403 Forbidden

- 6. Log Enforcement Action
 - Audit trail
 - Compliance tracking

1.7.4 Usage Examples

1.7.4.1 1. Create PEP

```
package main

import (
    "github.com/mauriciomferz/Gauth_go/pkg/gauth"
)

func main() {
    // Create token validator
    tokenValidator := gauth.NewJWTValidator(secretKey)

    // Create PDP
    pdp := gauth.NewSimplePDPWithPAP(pap)

    // Create audit logger
    auditLogger := gauth.NewFileAuditLogger("/var/log/gauth/enforcement.log")

    // Create compliance tracker
    complianceTracker := gauth.NewComplianceTracker()

    // Create PEP
    pep := gauth.NewPowerEnforcementPoint(
        tokenValidator,
        pdp,
        auditLogger,
        complianceTracker,
        "strict", // enforcement mode
    )

    fmt.Println(" PEP initialized")
}
```

1.7.4.2 2. Enforce Authorization (HTTP Middleware)

```
// HTTP middleware for authorization enforcement
func AuthorizationMiddleware(pep *gauth.PowerEnforcementPoint) gin.HandlerFunc {
    return func(c *gin.Context) {
        // Extract token from Authorization header
    }
}
```

```

token := c.GetHeader("Authorization")
if token == "" {
    c.JSON(403, gin.H{"error": "No authorization token"})
    c.Abort()
    return
}

// Remove "Bearer " prefix
token = strings.TrimPrefix(token, "Bearer ")

// Create enforcement request
request := &gauth.EnforcementRequest{
    Token:    token,
    Resource: c.Request.URL.Path,
    Action:   c.Request.Method,
    Context:  map[string]interface{}{
        "ip_address": c.ClientIP(),
        "user_agent": c.GetHeader("User-Agent"),
        "timestamp":  time.Now(),
    },
}

// Enforce authorization
decision, err := pep.Enforce(c.Request.Context(), request)
if err != nil {
    c.JSON(500, gin.H{"error": "Enforcement error"})
    c.Abort()
    return
}

if decision.Decision != "allow" {
    c.JSON(403, gin.H{
        "error": "Access denied",
        "reason": decision.Reason,
    })
    c.Abort()
    return
}

// Store decision in context for downstream use
c.Set("authz_decision", decision)
c.Next()
}

}

// Use middleware
router := gin.Default()
router.Use(AuthorizationMiddleware(pep))

```

1.7.4.3 3. Advisory Mode (Non-Blocking)

```
// Create PEP in advisory mode (logs but doesn't block)
pep := gauth.NewPowerEnforcementPoint(
    tokenValidator,
    pdp,
    auditLogger,
    complianceTracker,
    "advisory", // non-blocking mode
)

// In advisory mode:
// - All requests are allowed through
// - Decisions are logged for analysis
// - Helps with testing new policies before enforcement
```

1.7.4.4 4. Compliance Tracking

```
// Get compliance metrics
metrics := pep.GetComplianceMetrics()

fmt.Printf("Enforcement Metrics:\n")
fmt.Printf("  Total Requests: %d\n", metrics.TotalEnforcements)
fmt.Printf("  Allowed: %d\n", metrics.AllowedCount)
fmt.Printf("  Denied: %d\n", metrics.DeniedCount)
fmt.Printf("  Violations: %d\n", metrics.ViolationCount)

// Get violation details
violations := pep.GetViolations(time.Now().Add(-24*time.Hour), time.Now())
for _, v := range violations {
    fmt.Printf("Violation: %s attempted %s on %s\n",
        v.Subject, v.Action, v.Resource)
}
```

1.7.5 REST API Integration

```
# Protected API endpoint
GET /api/protected/resource
Authorization: Bearer <JWT_TOKEN>

# PEP intercepts and validates:
# 1. Token is valid
# 2. PDP allows access
# 3. Returns resource or 403
```

1.7.6 Implementation Details

- **Location:** [pkg/gauth/pep.go](#)
- **Components:** tokenValidator, pdp, auditLogger, complianceTracker

- **Modes:** strict (blocking), advisory (logging only)
-

1.8 Integration Examples

1.8.1 Complete E2E Authorization Flow

```
package main

import (
    "context"
    "fmt"
    "github.com/mauriciomferz/Gauth_go/pkg/gauth"
)

func main() {
    ctx := context.Background()

    // 1. Initialize P*P Components
    pap := gauth.NewPowerAdministrationPoint("pap-001", "Main PAP", "")
    pdp := gauth.NewSimplePDPWithPAP(pap)
    pipClient := pip.NewClient(pip.Config{
        RegistryURL: "http://localhost:8080",
    })
    pvpClient := gauth.NewMockPVPClient(false)

    tokenValidator := gauth.NewJWTValidator(secretKey)
    auditLogger := gauth.NewFileAuditLogger("/var/log/gauth/audit.log")
    complianceTracker := gauth.NewComplianceTracker()

    pep := gauth.NewPowerEnforcementPoint(
        tokenValidator,
        pdp,
        auditLogger,
        complianceTracker,
        "strict",
    )

    // 2. Create Authorization Policy (PAP)
    createRequest := &gauth.PolicyCreateRequest{
        PolicyType:      gauth.PolicyTypePoA,
        PolicyName:      "Healthcare Access Policy",
        ClientOwner:     "hospital-001",
        OwnersAuthorizer: "ceo-001",
        PolicyRules: gauth.PolicyRules{
            AllowedActions: []string{"read", "write"},
            ResourcePatterns: []string{"/api/patients/*"},
        },
    }
```



```

    },
    Scope: &gauth.PolicyScope{
        Countries: []string{"US", "CA"},
        Sectors:    []string{"healthcare"},
    },
}

policy, err := pap.CreatePolicy(ctx, createRequest)
if err != nil {
    log.Fatal(err)
}

// 3. Activate Policy
err = pap.ActivatePolicy(ctx, policy.PolicyID, "approver-cto")
if err != nil {
    log.Fatal(err)
}
fmt.Println(" Policy activated")

// 4. Verify Identity (PVP)
identityProof, err := pvpClient.VerifyIdentity(ctx, gauth.IdentityVerificationRequest{
    SubjectID:      "doctor-alice",
    IdentityType:   gauth.IdentityTypeNaturalPerson,
    ProofMethod:    gauth.ProofMethodEIDAS,
    Jurisdiction:   "DE",
    RequiredLevel:  "high",
})
fmt.Printf(" Identity verified: %v\n", identityProof.Verified)

// 5. Retrieve Attributes (PIP)
subjectAttrs, err := pipClient.GetSubjectAttributes(ctx, "doctor-alice")
fmt.Printf(" Attributes retrieved: roles=%v\n", subjectAttrs["roles"])

// 6. Make Authorization Decision (PDP)
decisionRequest := &gauth.AuthorizationDecisionRequest{
    Subject: "doctor-alice",
    Resource: "/api/patients/12345",
    Action:  "read",
    Context: map[string]interface{}{
        "ip_address": "192.168.1.100",
        "department": "cardiology",
    },
}

decision, err := pdp.MakeDecision(ctx, decisionRequest)
if err != nil {
    log.Fatal(err)
}

```

```

fmt.Printf(" PDP Decision: %s (reason: %s)\n", decision.Decision, decision.Reason)

// 7. Enforce Decision (PEP)
enforcementRequest := &gauth.EnforcementRequest{
    Token:    "jwt-token-here",
    Resource: "/api/patients/12345",
    Action:   "read",
    Context:  map[string]interface{}{
        "ip_address": "192.168.1.100",
    },
}

enforcementDecision, err := pep.Enforce(ctx, enforcementRequest)
if err != nil {
    log.Fatal(err)
}

if enforcementDecision.Decision == "allow" {
    fmt.Println(" Access GRANTED - Request forwarded")
    // Forward request to actual resource
} else {
    fmt.Println(" Access DENIED - Request blocked")
    // Return 403 Forbidden
}

// 8. Get Compliance Metrics
stats, _ := pap.GetPolicyStatistics(ctx)
fmt.Printf("\n System Status:\n")
fmt.Printf("   Active Policies: %d\n", stats.ActivePolicies)
fmt.Printf("   Total Enforcements: %d\n", complianceTracker.GetTotalEnforcements())
}

```

1.9 Best Practices

1.9.1 1. Policy Management

- Always validate policies before activation
- Use versioning for policy changes
- Tag policies for easy search and organization
- Set expiration dates for temporary policies
- Document policy changes in changelog
- Test in draft mode before activating

1.9.2 2. Decision Making

- Cache PDP decisions for performance
- Use distributed PDP for high availability

- **Implement circuit breakers** for external PIP calls
- **Monitor decision latency** (target < 5ms)
- **Log all authorization decisions** for audit

1.9.3 3. Enforcement

- Use **strict mode** in production
- **Test with advisory mode** first
- **Implement rate limiting** to prevent abuse
- **Log enforcement violations** for security analysis
- **Monitor compliance metrics** regularly

1.9.4 4. Integration

- **Initialize PIP** before PDP for attribute enrichment
- Use **PVP** for identity verification in subscription flow
- **Connect PEP** to multiple PDPs for redundancy
- **Implement health checks** for all P*P components
- Use **consistent error handling** across components

1.9.5 5. Performance

- **Enable caching** at PDP and PIP levels
- Use **connection pooling** for external services
- **Implement async logging** for audit trails
- ****Monitor P*P latencies**** with metrics
- **Scale PDPs horizontally** for high load

1.9.6 6. Security

- **Rotate signing keys** regularly
- Use **TLS** for all P*P communication
- **Validate all inputs** at enforcement points
- **Implement replay protection** for tokens
- **Audit access** to PAP policy management

1.10 Quick Reference

1.10.1 Component URLs

Component	Development URL	Production URL
PAP	http://localhost:8080/api/v1/pap	Configure in production
PDP	http://localhost:8080/api/v1/pdp	Configure in production
PIP	http://localhost:8080/api/v1/pip	Configure in production
PVP	http://localhost:8080/api/v1/beta/pvp	Configure in production
PEP	Middleware in application	Middleware in application

1.10.2 Environment Variables

```
# Enable P*P features
GAUTH_RFC0111_ENABLED=1
GAUTH_USE_JWT_LIB=1

# PIP Configuration
GAUTH_PIP_ENABLED=true
GAUTH_PIP_CACHE_TTL=300s

# PDP Configuration
GAUTH_PDP_CACHE_ENABLED=true
GAUTH_PDP_CACHE_TTL=300s

# PEP Configuration
GAUTH_PEP_MODE=strict # or "advisory"
GAUTH_PEP_AUDIT_ENABLED=true
```

1.10.3 Common Commands

```
# Start GAuth server with P*P features
GAUTH_DEV_INDEX=1 GAUTH_RFC0111_ENABLED=1 GAUTH_USE_JWT_LIB=1 go run ./cmd/web-server

# Test PAP policy creation
curl -X POST http://localhost:8080/api/v1/pap/policies \
  -H "Content-Type: application/json" \
  -d '{"policy_type":"poa","policy_name":"Test"}'

# Test PDP authorization
curl -X POST http://localhost:8080/api/v1/pdp/authorize \
  -H "Content-Type: application/json" \
  -d '{"subject":"user:alice","resource":"/api/data","action":"read"}'
```

1.11 Additional Resources

- [RFC-0111 Specification](#)
 - [API Documentation](#)
 - [PAP Implementation Tests](#)
 - [PDP Integration Guide](#)
 - [Phase 2A Quick Start](#)
-

Last Updated: November 17, 2025

Version: 1.0.0

Status: Production Ready